

פרויקט סיום – טכנולוגיות אינטרנט ופיתוח פול סטאק

DerbyUp – ניוושי כדורגל לארגונים | מגישים - ניר דהן , ליאב צרפתי | מרצה – אוהד אסולין | הגשה – אוגוסט 2026

מסמך אבטחה בסיסית

הקדמה

כיום אנו בעיצומה של מהפכת AI שבאה לידי ביטוי בהנגשת בניית הקוד לקהל רחב הרבה יותר בעולם. כתוצאה מכך, המידע של כל משתמש נגיש הרבה יותר, מה שדורש אבטחה הרבה יותר גבוהה מבעבר. במסמך זה נפרט את השיטות בהן אנחנו שומרים על אבטחת המידע באתר שלנו.

מודל האיום

#	גורם "מאיים"	מטרה	חומרה
1	עובד סקרן	לראות ניוושים של עמיתים לפני נעילה	בינוני
2	עובד תחרותי	לשנות לעצמו נקודות, לשנות ניוושי בדיעבד, או לנחש אחרי הפתיחה	גבוה
3	עובד שנוגע ב-DevTools	לעקוף ולידציות של הדפדפן	גבוה
4	גורם חיצוני	גישה לנתוני הארגון	גבוה
5	סורק אוטומטי	ניצול חולשות גנריות	בינוני
6	מתפעל מודל שפה	Prompt injection - לגרום ליועץ לחרוג מתפקידו או לחשוף את הוראותיו	בינוני
7	משתמש שמנסה לשנות הרשאות	לסמן את עצמו כמנהל אתר	גבוה

Authentication - איך מתבצע האימות

המנגנון

Supabase Auth עם אימייל וסיסמה.

פעולות	שלב
Supabase מגבב את הסיסמה ב- bcrypt ושומר ב- auth.users	הרשמה
mailer_autoconfirm פעיל - החשבון פעיל מיד, בלי קישור אימות	
טריגר on_auth_user_created יוצר פרופיל (0 נקודות - אין מענק)	
אימות מול JWT	התחברות
ה-JWT ב- sameSite · secure · httpOnly cookie	שמירת session
middleware.ts מרענן את ה-token בכל בקשה	רענון
ביטול ה-session ומחיקת ה-cookie	התנתקות

Authorization - איך מתבצעות ההרשאות

ארבע דרגות למשתמש

רמה	מי	למה יכול להגיע
אורח	לא מחובר	דף נחיתה, הרשמה, התחברות
משתמש רשום	עובד	הליגות שלו, ניוושי, אתגר, יועץ, פרופיל
אדמין ליגה	leagues.creator_id	לנהל רק את הליגה שיצר
מנהל אתר	profiles.is_site_admin	admin/ - כל משתמש, משחק וליגה; עיבוד, מינוי ומחיקה

שתי שכבות אכיפה - הגנה לעומק

```
request
|
+-- Layer 1: middleware.ts      -> signed in? admin? else redirect
+-- Layer 2: Server Action      -> validation + ownership check
|
v
+-- Layer 3: RLS in Postgres    -> the real line of defence
```

למה שלוש שכבות: שכבות 1–2 הן חוויית משתמש והגנה ראשונה. שכבה 3 היא הביטחון האמיתי - גם אם יכתב באג באפליקציה שמדלג על בדיקה, ה-DB עדיין לא יחזיר את השורות.

RLS - הליבה

```
alter table predictions      enable row level security;
alter table profiles         enable row level security;
alter table leagues          enable row level security;
alter table league_members   enable row level security;
alter table notifications    enable row level security;
alter table puzzle_attempts  enable row level security;
alter table user_achievements enable row level security;
alter table advisor_insights enable row level security;
alter table advisor_daily_pick enable row level security;
alter table advisor_conversations enable row level security;
alter table advisor_messages enable row level security;
alter table advisor_usage    enable row level security;
alter table api_cache        enable row level security;
```

ניחוסים - קריאה ויצירה של הבעלים בלבד, ללא שינוי או מחיקה:

```
create policy "predictions_select_own" on predictions
  for select using (auth.uid() = user_id);

create policy "predictions_insert_own" on predictions
  for insert with check (auth.uid() = user_id);

-- No UPDATE and no DELETE policy:
-- a user cannot modify or remove a prediction directly.
-- Settlement runs as service role; cancellation goes through
-- a SECURITY DEFINER function.
```

ניסיונות באתגר - קריאה ויצירה של הבעלים; הניקוד נקבע בשרת:

```
create policy "attempts_select_own" on puzzle_attempts
  for select using (auth.uid() = user_id);
```

ליגות - רק ליגות שהמשתמש חבר בהן:

```
create policy "leagues_select_member" on leagues
  for select using (
    exists (select 1 from league_members
            where league_id = leagues.id and user_id = auth.uid())
  );

create policy "leagues_update_creator" on leagues
  for update using (auth.uid() = creator_id);
```

חברי ליגה - רואים רק חברים בליגות משותפות:

```
create policy "members_select_shared_league" on league_members
for select using (
    exists (select 1 from league_members m
            where m.league_id = league_members.league_id
            and m.user_id = auth.uid())
);
```

פרופילים - הפרופיל שלך, ופרופילים בליגות משותפות:

```
create policy "profiles_select_visible" on profiles
for select using (
    auth.uid() = id
    or exists (
        select 1 from league_members a
        join league_members b on a.league_id = b.league_id
        where a.user_id = auth.uid() and b.user_id = profiles.id
    )
);

create policy "profiles_update_own" on profiles
for update using (auth.uid() = id);
```

דף הנחיתה - החלון היחיד שנפתח לאורח

דף הנחיתה מציג שלושה משחקים אמיתיים עם הסיכויים וכרטיס יועץ, אך questions, games ו-advisor_daily_pick פתוחות ל-authenticated בלבד. אותה תבנית: המדיניות נשארת, ונפתחות שתי פונקציות צרות - landing_fixtures() ו-landing_advisor_card() - בלי פרמטרים בכלל, עם המכסה בתוך הגוף (שלוש שורות ואחת), בלי אף מזהה, וב-grant גם ל-anon. מה שנחשף לאורח - שמות קבוצות, מועד, תוצאה ומחירים - פומבי ממילא בכל אתר ספורט.

מניעת שינוי ניקוד והרשאות - נקודה קריטית

ל-profiles יש policy ל-UPDATE (לעריכת שם ואווטאר), ולכאורה משתמש יכול לשנות גם את total_points שלו - ומאז הוספת דרגת מנהל האתר, גם את is_site_admin. המניעה בטריגר:

```
create function prevent_score_tampering() returns trigger
language plpgsql as $$
begin
    if current_setting('role', true) = 'service_role' then return new; end if;

    if new.total_points is distinct from old.total_points
    or new.total_predictions is distinct from old.total_predictions
    or new.total_correct is distinct from old.total_correct then
        raise exception 'score columns cannot be modified directly';
    end if;

    if new.is_site_admin is distinct from old.is_site_admin
    and coalesce(current_setting('app.grant_site_admin', true), 'off') <> 'on' then
        raise exception 'site admin cannot be granted directly';
    end if;

    return new;
end; $$;

create trigger profiles_protect_score
before update on profiles for each row
execute function prevent_score_tampering();
```

המשתמש יכול לעדכן שם ואווטאר בלבד. עמודות הניקוד מתעדכנות אך ורק בעיבוד, ודגל המנהל זו רק דרך admin_set_site_admin() - שפותחת את הטריגר לטרנוקציה שלה בלבד, בודקת שהקורא מנהל, ואוסרת עליו לשנות את עצמו. כך תמיד נותר מנהל אחד לפחות, ומנהל אחד שנפרץ אינו יכול להדיח בשקט את כל השאר.

פעולות המותרות רק למשתמש מחובר

פעולה	אורח	משתמש מחובר	אדמין ליגה	מנהל אתר
צפייה בדף נחיתה	רשאי	רשאי	רשאי	רשאי
הרשמה / התחברות	רשאי	-	-	רשאי
צפייה במשחקים, כולל תוצאות חיות	לא רשאי	רשאי	רשאי	רשאי
הנחת ניחוש (כולל תוצאה מדויקת)	לא רשאי	רשאי	רשאי	רשאי
ביטול ניחוש (עד 10 דק')	לא רשאי	רשאי (רק שלו)	רשאי (רק שלו)	רשאי
יצירה / הצטרפות לליגה	לא רשאי	רשאי	רשאי	רשאי
צפייה בטבלת דירוג, כולל התוספת החיה	לא רשאי	רשאי (בליגות שלו)	רשאי	רשאי
אתגר יומי	לא רשאי	רשאי	רשאי	רשאי
שאלת היועץ	לא רשאי	רשאי (10 ליום)	רשאי	רשאי
בחירת משחקים לליגה	לא רשאי	לא רשאי	רשאי (בליגות שלו)	רשאי
סימון בחירת עורך	לא רשאי	לא רשאי	רשאי (בליגות שלו)	רשאי
עדכון פרסי הליגה	לא רשאי	לא רשאי	רשאי (בליגות שלו)	רשאי
עיבוד ידני בליגה	לא רשאי	לא רשאי	רשאי (בליגות שלו)	רשאי
גישה ל-admin - כל משתמש, משחק וליגה	לא רשאי	לא רשאי	לא רשאי	רשאי
מינוי מנהל אתר - מחיקת משתמש	לא רשאי	לא רשאי	לא רשאי	רשאי (לא על עצמו)
הרצת cron	לא רשאי	לא רשאי	לא רשאי	רשאי

מניעת גישה למידע של משתמש אחר

מנגנון 1 - RLS מסנן בשאלתה עצמה

הבקשה נשלחת עם ה-JWT של המשתמש. Postgres מוסיף את תנאי ה-policy לכל שאלתה.

```
select * from predictions // Run by User A
select * from predictions where auth.uid() = user_id // Run by PG
// Result: Only User A results
```

אין דרך לעקוף. גם `select * from predictions` ידני יחזיר רק את השורות המותרות. זו הגנה בשכבת הנתונים, לא באפליקציה.

מנגנון 2 - הנתונים לא עוזבים את השרת

Server Components מרנדרים בשרת ושולחים HTML. אין endpoint שמחזיר JSON גולמי, ואין payload שאפשר לפתוח ב-DevTools ולמצוא בו נתוני משתמש אחר. גם הטבלה החיה: השורות של `league_live_predictions` נקראות ומסוכמות בשרת, ומה שמגיע לדפדפן הוא הסך פר-חבר בלבד.

מנגנון 3 - בדיקת בעלות ב-Actions

כל Action שנוגעת במשאב מאמתת בעלות לפני הפעולה - גם אם RLS ממילא היה חוסם. זה מייצר הודעת שגיאה ברורה במקום תוצאה ריקה מבלבלת.

מקרה מיוחד: ניחושים לפני נעילה

עובד סקרן (איום #1) לא אמור לראות ניחושים של עמיתים לפני שהמשחק ננעל. במוצר הנוכחי אין בכלל מסך שמציג ניחושים של אחרים - הפיצ'ר לא קיים באופן מכוון. `predictions_select_own` מבטיח שגם ניסיון ישיר לא יחזיר דבר.

ולידציה של קלטים

שלוש שכבות

שכבה	מה נבדק	ניתן לעקיפה?	תפקיד
HTML	min ,type ,required	כן	UX
Zod בשרת	טיפוס, טווח, פורמט	לא	ההגנה
אילוץ DB	FK ,UNIQUE ,CHECK	לא	רשת אחרונה

בשרת - לא בלקוח

```
export async function makePrediction(input: unknown) {
  const parsed = makePredictionSchema.safeParse(input);
  if (!parsed.success) return { ok: false, error: 'קלט לא תקין' };
  // ...
}
```

הקלט מוגדר **unknown** בכוונה - שום דבר לא נחשב מהימן עד שעבר **parse**.

SQL Injection

לא רלוונטי בארכיטקטורה הזו. ה-Supabase client בונה שאילתות פרמטריות; אין שרשור מחרוזות לתוך SQL. הפונקציות מקבלות פרמטרים מוקלדים.

XSS

React מסמן (escapes) כל טקסט אוטומטית. `<script>alert(1)</script>` בשם תצוגה יוצג כטקסט ולא יורץ. `dangerouslySetInnerHTML` אינו בשימוש בשום מקום בפרויקט - זה כלל מפורש.

CSRF

Server Actions מוגנות מובנית: Next.js מאמת origin ומשתמש ב-tokens חד-פעמיים. ה-cookies מוגדרים `.sameSite`.

הגנה על קריאות ל-API

Server Actions

הגנה	מימוש
רצות בשרת בלבד	הקוד לא נשלח ללקוח
אימות session	כל Action מאמתת בתחילתה
ולידציה	Zod על כל קלט
בדיקת בעלות	לפני כל פעולה על משאב
CSRF	מובנה ב-Next.js

Route Handlers של cron

```
export async function POST(req: Request) {
  const auth = req.headers.get('authorization');
  if (auth !== `Bearer ${process.env.CRON_SECRET}`) {
    return new Response('Unauthorized', { status: 401 });
  }
}
```

היועץ - הגנה בשלוש שכבות

מודל שפה מוסיף מחלקת איום מיוחד למוצר: הקלט הוא טקסט חופשי שמגיע למודל. האיומים הגדולים של מצב זה:

שכבה	היכן	עלות	מה נעצר
כללים	lib/advisor/guard.ts	0	ריק · ארוך מדי · תווי בקרה · URL · תבניות injection בעברית ובאנגלית
מסווג	lib/advisor/classifier.ts	כ-0.00006\$	נושא אחר · נושאי כדורגל שאינם קשורים למשחק הנוכחי · מניפולציה
סכמה	lib/advisor/schema.ts	0	פלט שלא עומד בסכמה · המלצה על אפשרות שאינה קיימת במשחק

שמירת secrets

סיווג

משתנה	סיווג	חשוף ללקוח?
NEXT_PUBLIC_SUPABASE_URL	ציבורי	כן
NEXT_PUBLIC_SUPABASE_ANON_KEY	ציבורי - כפוף ל-RLS	כן
SUPABASE_SERVICE_ROLE_KEY	סוד - עוקף RLS וטריגרים	לא
FOOTBALL_API_KEY	סוד	לא
CRON_SECRET	סוד	לא
GEMINI_API_KEY	סוד	לא - נקרא רק ב-Server cron Actions
GEMINI_MODEL · GEMINI_GUARD_MODEL	תצורה	לא
ADVISOR_DAILY_LIMIT	תצורה - נאכף בשרת, ב-Postgres	לא
LIVE_SCORES_ENABLED	תצורה - מפסק לסנכרון החי	לא

כללי השמירה על הסיווג בפרויקט שלנו

1. `env*` ב-`gitignore`. מהקומיט הראשון
2. סודות מוגדרים ב-`Vercel Environment Variables` בלבד
3. רק משתנים ציבוריים באמת מקבלים תחילת `NEXT_PUBLIC_`
4. `SUPABASE_SERVICE_ROLE_KEY` מיובא אך ורק ב-`lib/supabase/admin.ts`, שנקרא רק מ-Route Handlers של `cron` - לעולם לא מ-Server Component או Action של משתמש
5. `env.example`. בריפו עם שמות בלבד, בלי ערכים.

סיכונים שעדיין קיימים

#	סיכון	חומרה	למה נותר	מה היינו עושים
1	אין אימות אימייל ו-FA2	בינוני	מגבלת הגרסה החינמית של Supabase	מעבר לגרסה בתשלום + שירות אימות (Resend)
2	סיסמה 8 תווים בלי דרישת מורכבות	נמוך	איוון מול UX. מינימום 8 נאכף גם ב-Supabase	שימוש בשירות עזר
3	אין הצפנה ברמת השדה	נמוך	אין מידע רגיש במיוחד	pgcrypto לפי צורך
4	אין מחיקת חשבון בידי המשתמש	בינוני	החלטה מכוונת - למנוע "איפוס" נתונים; מנהל אתר יכול למחוק	מחיקה עצמית + ייצוא נתונים

#	סיכון	חומרה	למה נותר	מה היינו עושים
5	אין הסתרה מהלידרבורד הכללי	בינוני	החלטה מכוונת - הלוח מיועד לכלול את כולם	דגל is_public בפרופיל, או הסתרה ברמת הארגון
6	Prompt injection יצירתי שאינו נתפס בדפוסים	בינוני	שכבה 1 מבוססת דפוסים, ושכבה 1ב היא מודל שיכול לטעות	הרחבת הדפוסים + דגימת שיחות שנחסמו
7	אין rate limiting מעבר למכסת היועץ	נמוך	המכסה חוסמת את הנזק הכספי ; לא את הרעש	Redis Upstash על ה-Actions